

Практическое занятие 26: Добавление и редактирование данных через админ-панель

Цель занятия:

- Научиться добавлять и редактировать данные через админ-панель Django.
 - Реализовать возможность добавления главных фото и документов (PDF или DOCX) к мероприятиям.
 - Настроить отображение загруженных файлов в админ-панели и на сайте.
-

Задачи занятия

1. Введение в добавление файлов через админ-панель
 2. Настройка модели для загрузки изображений и документов
 3. Настройка админ-панели для работы с файлами
 4. Настройка обработки медиа-файлов в Django
 5. Отображение загруженных файлов на сайте
 6. Выполнение практического упражнения. Добавление мероприятий с файлами через админ-панель
 7. Выполнение дополнительных заданий. Ограничение типов, добавление миниатюр
-

Задача 1: Введение в добавление файлов через админ-панель

1.1. Возможности админ-панели по работе с файлами

- **Админ-панель Django** позволяет загружать и управлять файлами, связанными с моделями.
- **ImageField** и **FileField**: Поля моделей Django, предназначенные для хранения ссылок на загруженные изображения и файлы.
- **Медиа-файлы**: Требуют особой настройки в проекте Django для правильной работы.

1.2. Что мы будем реализовывать

- **Добавление главного изображения** к каждому мероприятию.

- **Добавление документа** (PDF или DOCX) с дополнительной информацией о мероприятии.
 - **Настройка админ-панели** для удобного управления этими файлами.
 - **Отображение файлов** на сайте для пользователей.
-

Задача 2: Настройка модели для загрузки изображений и документов

2.1. Обновление модели Event

Шаги:

1. Откройте файл `events/models.py`.
2. Импортируйте необходимые модули:

```
from django.db import models
from django.utils import timezone
```

3. Добавьте поля для изображения и документа в модель `Event` :

```
# events/models.py

class Event(models.Model):
    # ... существующие поля ...
    title = models.CharField(max_length=200)
    description = models.TextField()
    date = models.DateTimeField()
    location = models.CharField(max_length=200)
    category = models.ForeignKey(Category, on_delete=models.CASCADE)

    # Новые поля
    main_image = models.ImageField(upload_to='event_images/',
                                   blank=True, null=True)
    document = models.FileField(upload_to='event_documents/',
                                blank=True, null=True)

    def __str__(self):
        return self.title
```

Пояснение:

- `main_image` : Поле для загрузки изображения. `upload_to` определяет папку внутри `MEDIA_ROOT` , куда будут сохраняться изображения.

- `document` : Поле для загрузки файлов. Поддерживает любые типы файлов.
`upload_to` определяет папку для документов.

2.2. Применение миграций

После изменения модели необходимо создать и применить миграции.

Шаги:

1. Создайте миграции:

```
python manage.py makemigrations
```

2. Примените миграции:

```
python manage.py migrate
```

Пояснение:

- Эти команды обновят структуру базы данных, добавив новые поля к таблице `Event`.

Задача 3: Настройка админ-панели для работы с файлами

3.1. Обновление `EventAdmin` для отображения новых полей

Шаги:

1. Откройте файл `events/admin.py`.
2. Импортируйте необходимые модули:

```
from django.contrib import admin
from .models import Event, Category
```

3. Обновите класс `EventAdmin` :

```
# events/admin.py

class EventAdmin(admin.ModelAdmin):
```

```

list_display = ('title', 'date', 'location', 'category',
'display_image')
list_filter = ('category', 'date')
search_fields = ('title', 'description')
ordering = ('date',)
readonly_fields = ('display_image_preview',)

fieldsets = (
    (None, {
        'fields': ('title', 'description', 'date', 'location',
'category')
    }),
    ('Media', {
        'fields': ('main_image', 'display_image_preview',
'document')
    }),
)

def display_image(self, obj):
    if obj.main_image:
        return 'Да'
    return 'Нет'
display_image.short_description = 'Изображение'

def display_image_preview(self, obj):
    if obj.main_image:
        return format_html('', obj.main_image.url)
    return 'Нет изображения'
display_image_preview.short_description = 'Предпросмотр
изображения'

```

Пояснение:

- `display_image`: Метод, который отображает, есть ли у мероприятия изображение.
- `display_image_preview`: Метод для отображения предпросмотра изображения в админ-панели.
- `readonly_fields`: Поля, которые нельзя редактировать, но можно просматривать.
- `fieldsets`: Организует поля на странице редактирования в группы.

3.2. Импорт `format_html`

Не забудьте импортировать `format_html` для безопасного отображения HTML в админ-панели.

```

from django.utils.html import format_html

```

3.3. Сохранение и проверка

- Сохраните файл `admin.py`.
 - Перейдите в админ-панель и попробуйте добавить или отредактировать мероприятие.
 - Убедитесь, что можете загружать изображения и документы.
 - Проверьте, что предпросмотр изображения отображается корректно.
-

Задача 4: Настройка обработки медиа-файлов в Django

4.1. Настройка `MEDIA_URL` и `MEDIA_ROOT`

Шаги:

1. Откройте файл `settings.py` вашего проекта.
2. Добавьте в конец файла следующие настройки:

```
# settings.py

import os

MEDIA_URL = '/media/'
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
```

Пояснение:

- `MEDIA_URL` : URL, по которому будут доступны медиа-файлы.
- `MEDIA_ROOT` : Путь на сервере, где будут храниться загруженные файлы.

4.2. Настройка URL для медиа-файлов

Шаги:

1. Откройте файл `urls.py` вашего проекта (не приложения).
2. Добавьте импорт и настройки для обработки медиа-файлов в режиме разработки:

```
# your_project/urls.py

from django.contrib import admin
from django.urls import path, include
```

```
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('events.urls')),
]

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
        document_root=settings.MEDIA_ROOT)
```

Пояснение:

- `static()` : Позволяет Django обслуживать медиа-файлы во время разработки при включенном режиме `DEBUG` .

4.3. Проверка настроек

- Перезапустите сервер разработки:

```
python manage.py runserver
```

- Убедитесь, что можете загружать файлы через админ-панель и что они сохраняются в директорию `media/` .

Задача 5: Отображение загруженных файлов на сайте

5.1. Обновление шаблона `event_detail.html`

Шаги:

1. Откройте файл `event_detail.html` в директории `templates/events/` .
2. Добавьте отображение изображения и документа:

```
<!-- events/templates/events/event_detail.html -->

{% extends 'base.html' %}

{% block title %}{{ event.title }}{% endblock %}

{% block content %}
```

```

<h1>{{ event.title }}</h1>
<p><strong>Дата и время:</strong> {{ event.date|date:"d.m.Y H:i" }}
</p>
<p><strong>Место:</strong> {{ event.location }}</p>
<p><strong>Категория:</strong> {{ event.category.name }}</p>
<p>{{ event.description }}</p>

{% if event.main_image %}
    
{% endif %}

{% if event.document %}
    <p><a href="{{ event.document.url }}">Скачать дополнительную
информацию</a></p>
{% endif %}

<!-- Остальной код -->
{% endblock %}

```

Пояснение:

- `{{ event.main_image.url }}`: Ссылка на загруженное изображение.
- `{{ event.document.url }}`: Ссылка на загруженный документ.

5.2. Обновление шаблона `event_list.html` (опционально)

Вы можете добавить отображение миниатюры изображения в списке мероприятий.

Шаги:

1. Откройте файл `event_list.html` в директории `templates/events/`.
2. Обновите код цикла по мероприятиям:

```

<!-- events/templates/events/event_list.html -->

{% extends 'base.html' %}

{% block title %}Список мероприятий{% endblock %}

{% block content %}
    <h1>Список мероприятий</h1>
    <ul>
        {% for event in events %}
            <li>
                {% if event.main_image %}
                    
    {% endif %}
    <a href="{% url 'events:event_detail' event.id %}">{{
event.title }}</a> -
    {{ event.date|date:"d.m.Y H:i" }} -
    {{ event.location }}
</li>
{% empty %}
    <li>Нет доступных мероприятий.</li>
{% endfor %}
</ul>
{% endblock %}

```

5.3. Проверка работы

- Перейдите на страницу детали мероприятия и убедитесь, что изображение и ссылка на документ отображаются.
- Перейдите на страницу списка мероприятий и проверьте отображение миниатюр (если добавляли).

Задача 6: Обновление галереи

6.1. Добавление ссылки на список мероприятий

В файле `events/views.py` добавьте следующее:

```

# events/views.py

from django.shortcuts import render
from .models import Event

def gallery(request):
    return render(request, 'events/gallery.html', {'events': events})

```

Пояснение:

- Импортируем модель `Event`.
- Получаем все мероприятия с помощью `Event.objects.all()` выше.
- Передаём список мероприятий в шаблон `gallery.html`.

6.2. Создание или обновление шаблона `gallery.html`

В директории `templates/events/` создайте файл `gallery.html` со следующим содержанием:

```
<!-- events/templates/events/gallery.html -->

{% extends 'base.html' %}

{% block title %}Галерея{% endblock %}

{% block content %}
    <h1>Галерея</h1>
    {% for event in events %}
        {% if event.main_image %}
            
        {% endif %}
    {% endfor %}
{% endblock %}
```

Пояснение:

- Проходимся по всем мероприятиям с помощью цикла `{% for event in events %}`.
- Проверяем, есть ли у мероприятия изображение: `{% if event.main_image %}`.
- Отображаем изображение с помощью ``.

6.3. Добавление маршрута для галереи.

В файле `events/urls.py` добавьте/измените маршрут:

```
# events/urls.py

from django.urls import path
from . import views

app_name = 'events'

urlpatterns = [
    # ... другие маршруты ...
    path('gallery/', views.gallery, name='gallery'),
]
```

Пояснение:

- Добавляем путь `/gallery/`, который ссылается на представление `gallery`.

Задача 7: Практическое упражнение: Добавление мероприятий с файлами через админ-панель (10 минут)

Задание для студентов:

- **Добавьте новое мероприятие** через админ-панель, загрузив изображение и документ.
- **Проверьте** отображение загруженных файлов на сайте.
- **Отредактируйте мероприятие**, заменив изображение и документ на другие файлы.
- **Удалите файлы** из мероприятия и убедитесь, что сайт корректно отображает отсутствие файлов.

Цель упражнения:

- Закрепить навыки работы с загрузкой и отображением файлов в Django.
- Убедиться в корректности настроек медиа-файлов в проекте.

Дополнительное задание

1. Ограничение типов загружаемых файлов

Цель: Ограничить загрузку файлов только определенных типов (например, только PDF и DOCX для документов).

Шаги:

1. Установите библиотеку `django-cleanup` (опционально):

```
pip install django-cleanup
```

Пояснение:

- `django-cleanup` автоматически удаляет старые файлы при обновлении или удалении модели.

2. Добавьте `FileExtensionValidator` к полю `document`:

```
# events/models.py

from django.core.validators import FileExtensionValidator

class Event(models.Model):
```

```
# ... существующие поля ...
document = models.FileField(
    upload_to='event_documents/',
    blank=True,
    null=True,
    validators=[FileExtensionValidator(allowed_extensions=['pdf',
'docx'])])
)
```

Пояснение:

- `FileExtensionValidator`: Валидатор, который проверяет расширение загружаемого файла.

3. Примените миграции (не обязательно, если вы только добавили валидатор):

```
python manage.py makemigrations
python manage.py migrate
```

4. Проверьте загрузку документов через админ-панель, попробуйте загрузить файл неподдерживаемого типа.

2. Добавление миниатюр изображений с помощью `django-imagekit`

Цель: Автоматически создавать миниатюры изображений для отображения на сайте.

Шаги:

1. Установите библиотеку `django-imagekit`:

```
pip install django-imagekit
```

2. Добавьте `imagekit` в `INSTALLED_APPS` в `settings.py`:

```
# settings.py

INSTALLED_APPS = [
    # ... другие приложения ...
    'imagekit',
]
```

3. Обновите модель `Event` для создания миниатюр:

```
# events/models.py

from imagekit.models import ImageSpecField
from imagekit.processors import ResizeToFill

class Event(models.Model):
    # ... существующие поля ...
    main_image = models.ImageField(upload_to='event_images/',
    blank=True, null=True)
    main_image_thumbnail = ImageSpecField(
        source='main_image',
        processors=[ResizeToFill(100, 50)],
        format='JPEG',
        options={'quality': 60}
    )
```

4. Обновите шаблоны для использования миниатюр:

```
<!-- events/templates/events/event_list.html -->

{% if event.main_image %}
    
{% endif %}
```

Пояснение:

- `ImageSpecField`: Позволяет создавать производные изображения на основе исходного.

Заключение

Поздравляем! Вы успешно:

- Научились добавлять и редактировать данные через админ-панель Django.
- Реализовали возможность загрузки изображений и документов в модели.
- Настроили админ-панель для удобной работы с файлами.
- Настроили обработку и отображение медиа-файлов в проекте Django.

Если у вас возникнут дополнительные вопросы или потребуется помощь, не стесняйтесь обращаться.

Вопросы для обсуждения

- **Как обеспечить безопасность при загрузке файлов пользователями?**
 - Использовать валидаторы для проверки типов и размеров файлов.
 - Ограничивать доступ к директориям с файлами.
 - Проверять файлы на наличие вредоносного кода.
 - **Почему важно правильно настроить `MEDIA_ROOT` и `MEDIA_URL` ?**
 - Чтобы Django знал, где хранить и как обслуживать медиа-файлы.
 - Неправильная настройка может привести к ошибкам при загрузке или отображении файлов.
 - **Какие дополнительные возможности предоставляет админ-панель для работы с файлами?**
 - Предпросмотр изображений.
 - Возможность удаления и замены файлов.
 - Использование сторонних пакетов для расширения функциональности.
-

Полезные ресурсы

- [Документация Django: Работа с файлами](#)
- [Документация Django: Настройка админ-панели](#)
- [django-imagekit](#)
- [django-cleanup](#)